

Guia Definitivo de Versionamento e Documentação de Software

Boas Práticas: Tags, Commits Semânticos, Releases e Changelogs

1. Introdução aos Conceitos de Histórico

Change (Mudança)

No contexto de desenvolvimento de software, uma **Change** representa qualquer alteração feita no código-fonte, nos arquivos de configuração ou na documentação do projeto. Toda evolução de um sistema é composta por um conjunto cumulativo de mudanças.

Log (Registro)

O **Log** é um registro cronológico e detalhado de eventos que ocorreram no sistema. No ecossistema Git, o **git log** funciona como a "caixa-preta" do repositório, documentando cada modificação exata, quem a realizou, o hash do commit e a data. É uma ferramenta voltada estritamente para auditoria técnica e desenvolvedores.

Changelog

Diferente do log técnico, o **Changelog** é um arquivo de texto estruturado (geralmente **CHANGELOG.md**) que resume de forma legível e organizada as mudanças significativas de um projeto entre versões específicas.

- **Público-alvo:** Usuários, gerentes de produto, equipes de suporte e outros desenvolvedores.
- **Objetivo:** Evitar que as pessoas tenham que ler centenas de commits para entender o que mudou. Ele filtra o ruído técnico e foca no valor entregue.

2. Gerenciamento de Ciclo de Vida: Git Tags e Commits de Release

Git Tag

Uma **Tag** no Git funciona como um marco ou "ponteiro fixo" em um ponto específico do histórico de commits. Enquanto as *branches* se movem conforme novos códigos são adicionados, uma tag permanece estática. Ela é utilizada primordialmente para identificar pontos de lançamento de versão (ex: **v1.0.0**, **v2.1.4**), seguindo as regras do **Versionamento Semântico (Semantic Versioning - SemVer)**:

- **MAJOR (Maior):** Quando há mudanças que quebram a compatibilidade com versões anteriores.
- **MINOR (Menor):** Quando novas funcionalidades são adicionadas sem quebrar a compatibilidade.
- **PATCH (Correção):** Quando apenas correções de bugs retrocompatíveis são implementadas.

Commits de Release

Um **Commit de Release** é o commit exato que consolida o fechamento de uma nova versão. Geralmente, ele é o responsável por atualizar o número da versão no arquivo de metadados do projeto (como **package.json**,

`pom.xml` ou `setup.py`) e por consolidar o `CHANGELOG.md` daquela iteração. É a partir desse commit específico que a **Git Tag** é criada e o pipeline de CI/CD dispara o deploy para produção.

3. Commits Semânticos (Conventional Commits)

O padrão de **Conventional Commits** (Commits Semânticos) introduz uma especificação leve sobre as mensagens de commit, facilitando a criação de ferramentas automatizadas para a geração de Changelogs. A estrutura básica segue o modelo: `<tipo>(escopo opcional): <descrição>`.

Abaixo estão os principais tipos utilizados e seus significados:

Tipo	Nome Extenso	Descrição e Aplicação Técnica
feat	Feature	Utilizado quando uma nova funcionalidade ou recurso inédito é adicionado ao sistema. Impacta diretamente o usuário final e altera a versão <i>Minor</i> no SemVer.
feature	Feature	Variante idêntica ao feat . Embora o padrão estrito do <i>Conventional Commits</i> recomende o uso de feat por brevidade, alguns times utilizam feature por preferência de clareza textual.
fix	Fix	Utilizado exclusivamente para a correção de um bug ou comportamento inesperado no software. Altera a versão <i>Patch</i> no SemVer.
refactor	Refactor	Utilizado para modificações que melhoram a estrutura interna do código , performance ou legibilidade, sem alterar o comportamento externo do sistema (não adiciona funcionalidade e não corrige bug).
docs	Documentation	Utilizado para alterações feitas estritamente na documentação do projeto , como o arquivo <code>README.md</code> , comentários explicativos de código, documentações de API (Swagger) ou manuais de arquitetura.

4. Exemplo Prático de Fluxo de Trabalho

Para visualizar a sinergia entre todos esses conceitos, observe o fluxo técnico abaixo:

- Desenvolvimento:** O desenvolvedor cria uma nova rota de login e realiza o commit: `feat(auth): adicionar suporte a autenticação por duas etapas (2FA)`
- Correção:** Um bug é encontrado na validação do token e corrigido: `fix(auth): corrigir expiração prematura do token de segurança`
- Documentação:** O manual da API é atualizado: `docs(api): atualizar guias de integração com o novo fluxo 2FA`
- Preparação da Release:** Ao fim do ciclo, cria-se o commit de release incrementando a versão para `v1.1.0` e atualizando o arquivo `CHANGELOG.md`.
- Tagging:** O comando `git tag -a v1.1.0 -m "Release v1.1.0"` é executado para fixar o marco na história do repositório.

Este ecossistema garante rastreabilidade, previsibilidade e profissionalismo no gerenciamento de qualquer produto digital.